



Twitch StreamSync

Every twitch feature you can dream of

Compatibility and Requirements

Unreal Engine Versions:

→ **5.0** (only WS), **5.1, 5.2, 5.3, 5.4, 5.5, and 5.6**

[Unsupported: Unreal Engine 5.0 since HTTPServer module is unavailable]

Supported Platforms:

- Windows 64-bit
- Architecture: Win64 only

Dependencies:

- Unreal's built-in *HTTP*, *WebSockets*, and *SSL* modules.

External Requirements:

- A Twitch Developer Application with a valid Client ID and Client Secret
- Redirect URI: <http://localhost:8080/Auth> for local OAuth loopback
- No external server is required unless you choose Webhook mode

Legal Notice / Disclaimer

This plugin and its author are not affiliated with, endorsed, or sponsored by Twitch Interactive, Inc.

All trademarks and logos are the property of their respective owners.

By using this plugin, you agree to comply with the **Twitch Developer Terms of Service**:

→ <https://dev.twitch.tv/docs/api/terms-of-service>

The plugin uses *Twitch's public APIs* strictly in accordance with *Twitch's rate limits and OAuth security requirements*. Users are responsible for ensuring their application remains compliant with Twitch's policies.

License

© 2025 Kybrien. All Rights Reserved.

This plugin and its documentation are provided as proprietary software under the Epic Games Marketplace Distribution Agreement. Redistribution or resale outside the Unreal Engine Marketplace is prohibited.

SUMMARY

1. Introduction.....	3
2. Basic Setup.....	4
3. Scopes.....	7
4. TwitchAuthentication.....	7
5. TwitchChat.....	9
6. EventSub.....	11
7. Helix : Update Stream (Async).....	12
8. Dev Simulator.....	13
9. Twitch Log Exporter.....	14
10. Webhook Transport (HTTPS callback).....	15
11. Refresh Token.....	16
12. Error Handling.....	17
13. Annexes.....	19

If you're more of a visual learner, I invite you to just open the BP_StreamSyncDEMO in the package, I commented EVERY node of the plugin so you could understand everything by yourself (check the doc just in case)

The BLUE commented nodes are the most important ones
The RED ones are the deprecated or non-recommended ones, that require good knowledge of some aspects of the connection to twitch

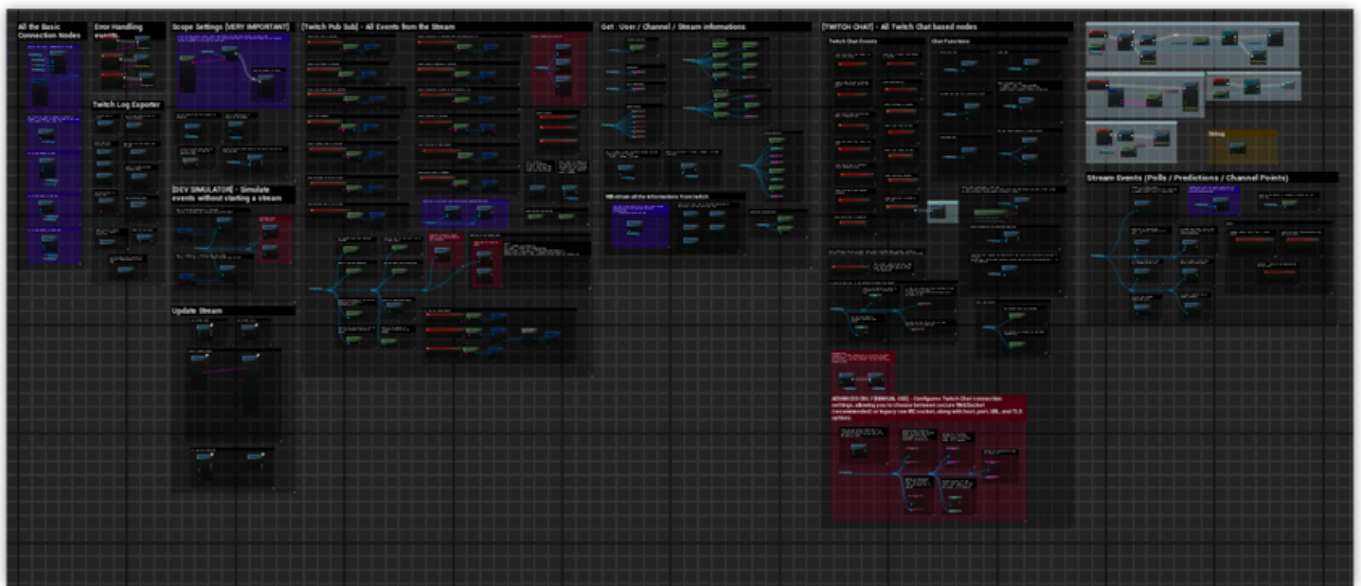
Twitch StreamSync

1. Introduction :

Twitch StreamSync is an all-in-one Unreal Engine plugin that connects your project to Twitch with minimal setup. It wraps **OAuth**, **Chat** (IRC), **EventSub** (WebSocket or Webhook), and **Helix** REST into Blueprint-first workflows and some focused C++ helpers.

Use it to ship real-time stream features : chat commands, channel point redeems, polls, predictions, raids, subs, bits (without rebuilding sockets), HTTP, or parsers.

This documentation is entirely made to help you understand and set up the Twitch plugin. We'll go through most of the components, the important nodes, the events and the potential troubleshootings you may encounter.



PS : I first wanted to do an BIG documentation to review all the nodes and possibilities, but it would be way too long and would over-complexify the documentation that i want clear, simple, and user-friendly (I did my best).

SOOO I thought I might do some tutorials aiming at precise in-game features, when uploaded I'll keep everyone in touch on my discord server.

Discord : <https://discord.gg/BwhyxQAAUn>

Ytb : <https://www.youtube.com/@AdriensDiscoveries>

Twitch StreamSync

Components Explanation

- **TwitchAuthentication**

Handles the Twitch login (OAuth), keeps your user token safe, and fetches your basic data (who you are, your channel, whether you're live etc.)

- **TwitchChat**

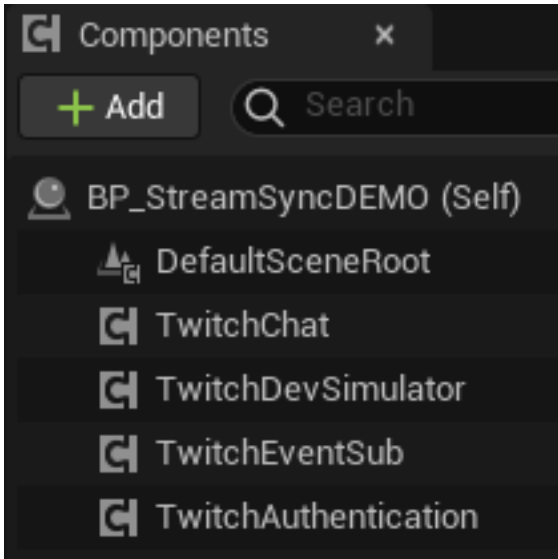
Connects to Twitch IRC over WebSocket and TLS, logs in with your token, sends messages, reads messages, and lets you register commands (like !hello). Also includes moderation helpers (slow mode, follower-only, clear chat, ban/timeout).

- **TwitchEventSub**

Receives Twitch EventSub notifications via WebSocket (no port needed) or Webhook (local HTTP + HMAC verification). Manages creating/listing/deleting subscriptions and raises callbacks like channel point redemption or prediction updated.

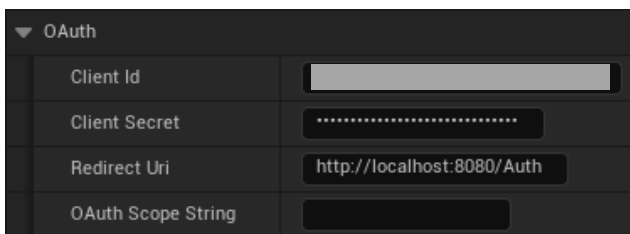
- **TwitchDevSimulator**

Built-in test bench that mimics Twitch events locally. Instead of going live (or asking chat to spam you), you can simulate events and chat messages to see how everything reacts.



2. Basic Setup :

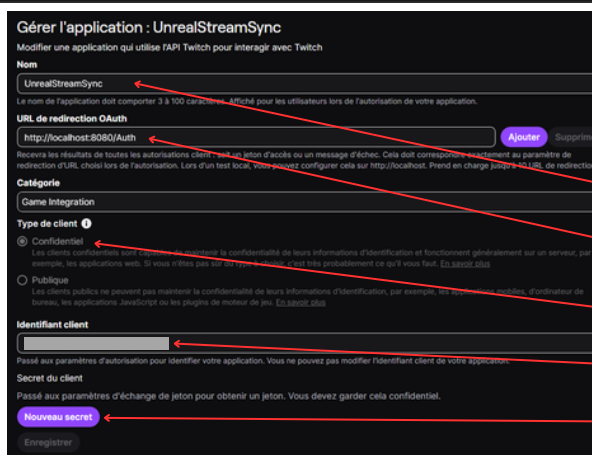
Step 1 - Credentials & defaults



First of all, add the components into your blueprint. For now we only need **TwitchAuthentication**. Let's fill it :

You will find the **Client id** and the **Secret Client** after registering your app/game on the twitch dev console.

The entire tutorial can be found on this page :
<https://dev.twitch.tv/docs/authentication/register-app>



Your twitch app/game name

Redirect Uri : <http://localhost:8080/Auth> (for WebSocket, if you want to use Webhook put your own URL)

Select : "Confidential" to get a secret Client

You will there get your Client ID for the OAuth Component

Generate a New Secret Client

Sorry for the french screenshot :(

Twitch StreamSync

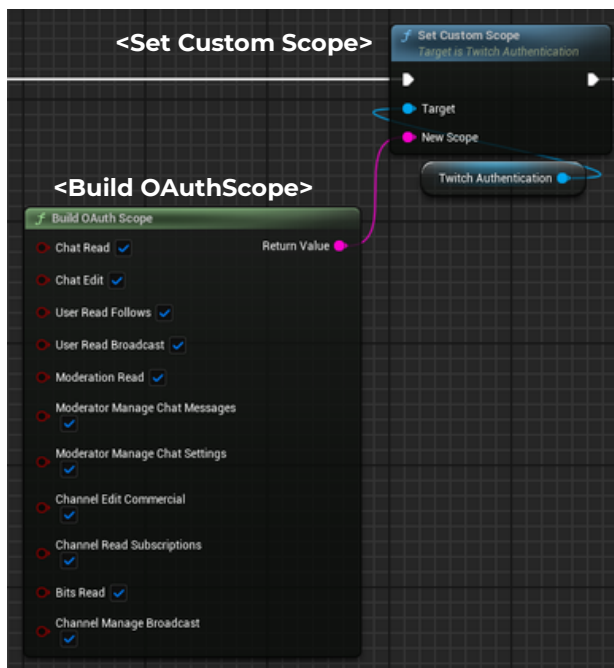
If you don't want to code any server (You're making a game without backend):

In the Redirect Uri of the component, write "**http://localhost:8080/Auth**" (same as the twitch app)
Why that ? It's a loopback redirect for local apps. After the user clicks "Authorize" on Twitch, the browser is sent back to your machine on port 8080 at path /Auth. Your plugin listens there, grabs the authorization code, and finishes the login
→ no public server needed.

In case you need a real serv and backend :

Refer to the Webhook part page 15..

Step 2 - Boot (two options)

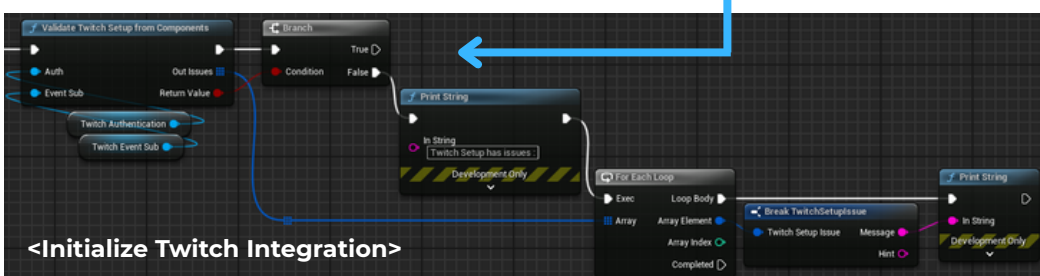


Whatever option you'll chose, first start by setting the custom scope you'll ask the player.

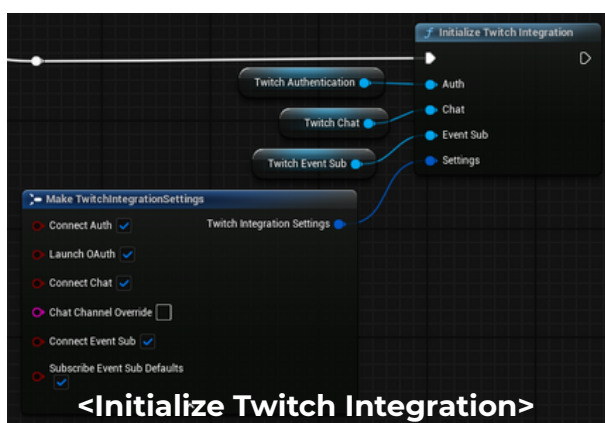
REMINDER : Depending on the scopes the streamer will accept, some actions may or may not be possible on our side.

Once you set up the scope, I recommend using "**Validate twitch Setup from comp**" to be sure everything is fine before trying to connect.

You can either replicate what i did in the screenshot below or do your own answer to the return being false



→ FIRST OPTION (Recommended)



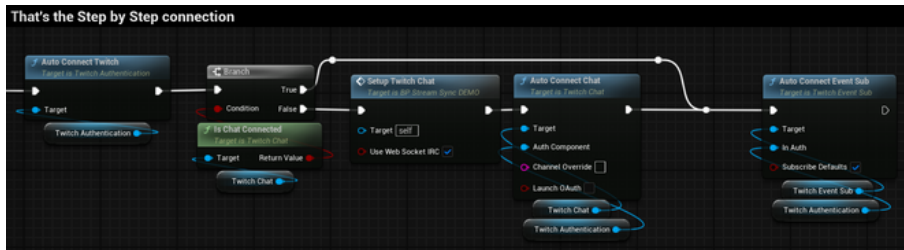
This option with the **All in One** node is my favourite since it does everything by itself. It (eventually) does the **Authentication**, connects to **Chat** and to the **EventSub** (based on the cases you check).

"Chat channel override" is useful if you want to listen or operate on another Streamer's chat WHILE still being connected to your current account (if you're a moderator for example).

→ It can also be used to send messages in #devsandbox (for example) in order to not pollute the twitch chat when doing your tests.

Twitch StreamSync

→ SECOND OPTION -----



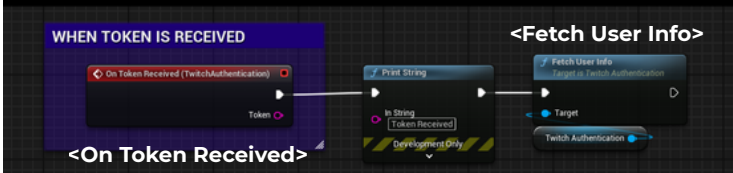
This option is the **Step by Step** connection to twitch. You'll see that to connect to Twitch, to the Chat or to EventSub you can simply call **"AutoConnect"<TheThingYouWant>**

Important Reminders :

- Keep in mind that if you want to call **"AutoConnectChat"** you should **"Setup Twitch Chat"** (i created this one in the BP) before -> to be sure the **"Use Web Socket IRC"** is set to **TRUE**. When set TRUE, it becomes Firewall-Friendly and doesnt need extra sockets code. (keep in **TRUE**)
- The bool **"Launch OAuth"** that you can find on some nodes is used to launch a new connection to twitch in your browser. I would suggest to keep it true since I implemented a token based code. It means that

Step 3 - Lifecycle

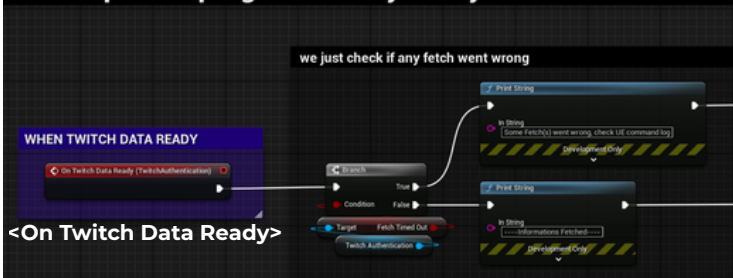
2nd Step - When Twitch sends us the token we fetch all the user's informations



Once you completed your connection to twitch with either the *All in One* or the *Step by Step* way, the flow will continue to the **"On Token Received"** event.

Once there, we want to **"Fetch User Info"** : it will give us his name, his pfp, the stream title etc. → Basically all the public informations you can get from twitch about the streamer (only informations, not events).

3rd Step -The plugin is finally ready



Last Step : The event **"On Twitch Data Ready"** will trigger when the plugin got everything he needed to do the basic work (get all the informations from the stream).



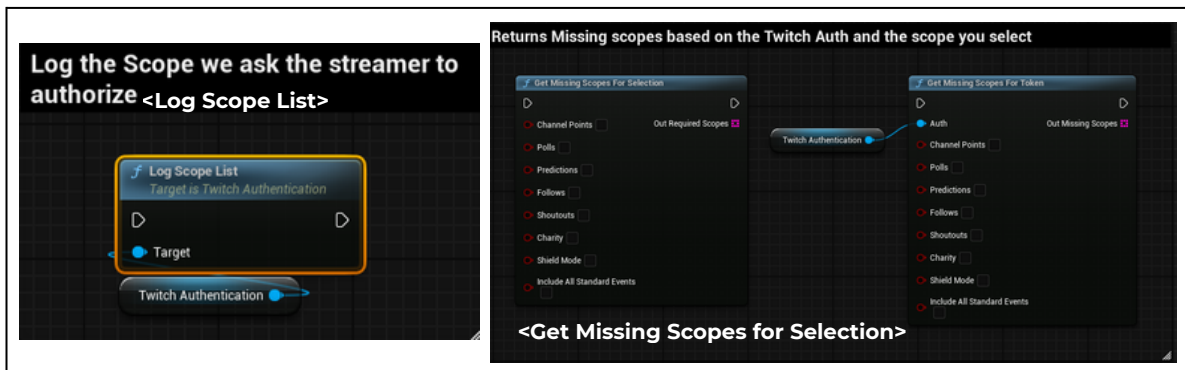
If you want to handle twitch chat and/or twitch events, refer to the parts about Twitch Chat (page 9), and Twitch EventSub (page 12) - BUT READ ALL THE NEXT STEPS FIRST

3. Scopes :

First of all, what are scopes ?

→ Basically, scopes are just **permissions** on the user Token. With the good permissions you're able to manage *chat / read chat / manage events* etc. (To modify your scope you can use the **Set Custom Scope** with the **Build OAuthScope** AND THEN launch a new Authentication)

[I created two nodes to make our life way easier, a **Log Scope** list and a **Get missing scope / Get Missing Scope for Token**

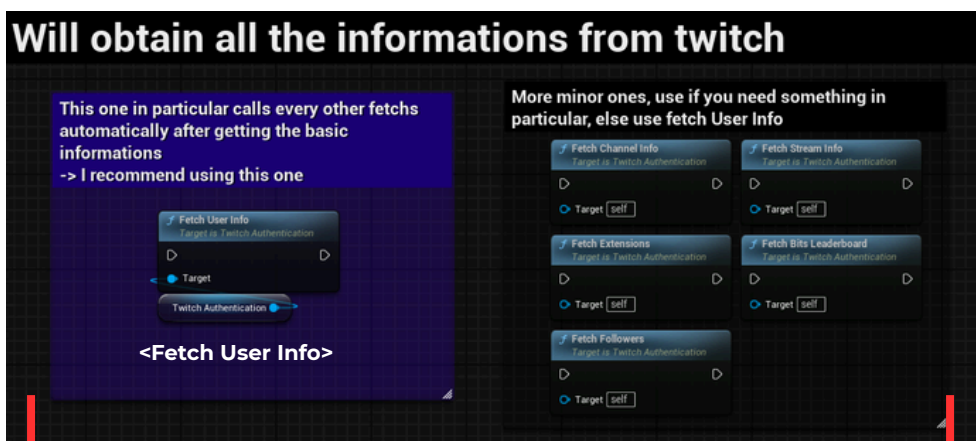


4. TwitchAuthentication :

1 - Role of TwitchAuthentication

Runs OAuth, stores the user token, validates scopes, and fetches data you can read from Blueprints. It's actually very simple, and to set it up you can go check the *Basic Setup* part (page 4).

2 - Fetchers (one-liners)



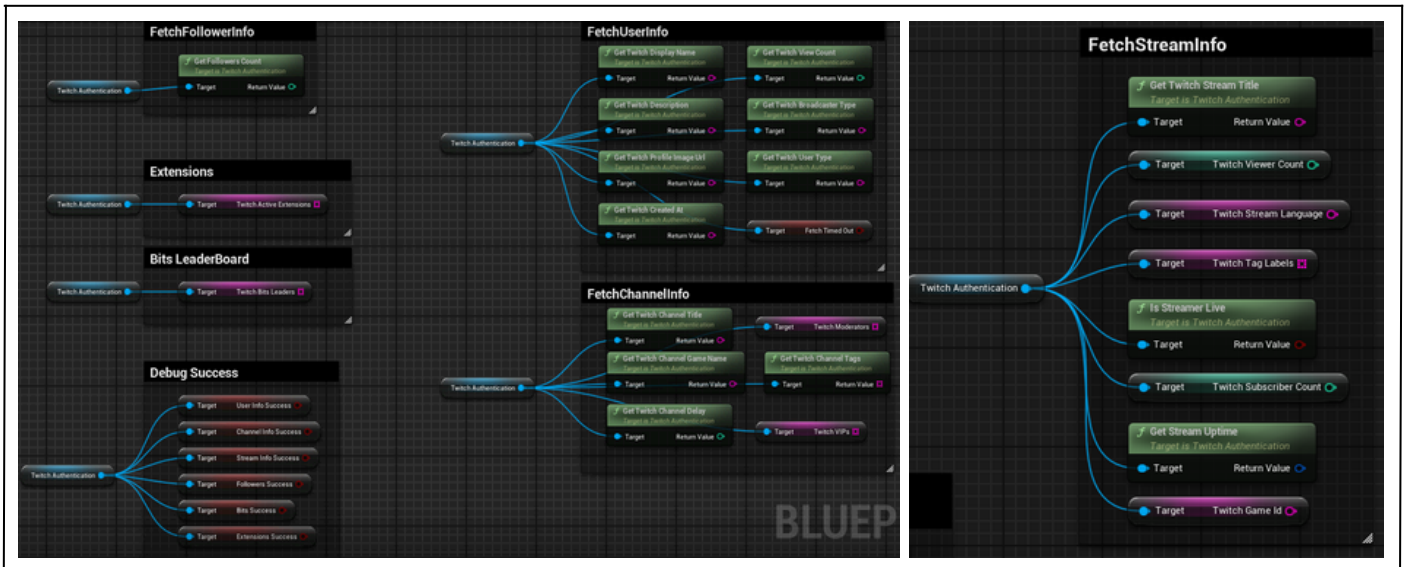
The Fetches nodes are used to get the informations from twitch with the help of the token.

Each fetch gets his part of the informations :

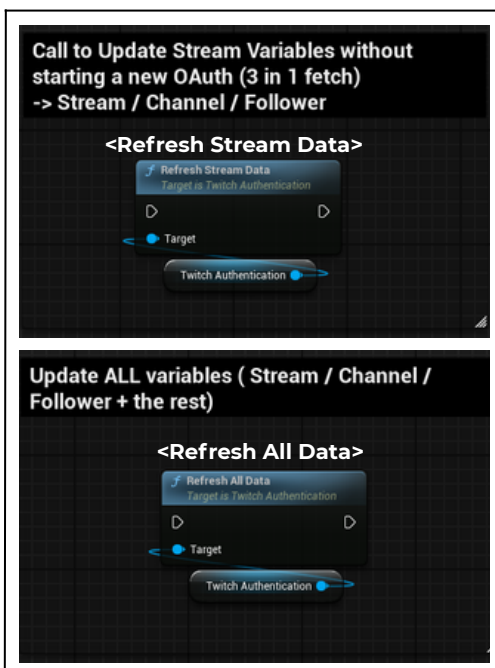
- **Fetch Channel Info**
- **Fetch Extensions**
- **Fetch Followers**
- **Fetch Stream Infos**
- **Fetch Bits Leaderboard**

And of course if you want an All in One node you can use **"Fetch User Info"** that will get all the informations by itself (That's what i recommend actually)

Twitch StreamSync



You can see just above all the informations you can get from the fetches. The **“Fetch User Info”** covers all of those categories at a the same time. The *individuals Fetchs* gives us different informations based on the one who chose (→ Refer to the screen above to know what Fetch covers what category). Of course you can use the **booleans** to check if the fetch went well.



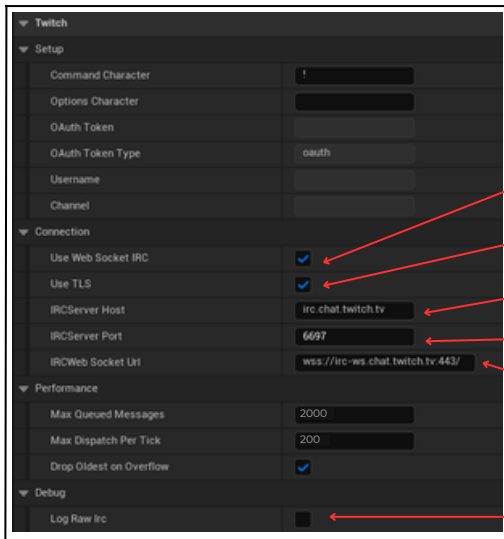
Once you fetched the wanted informations, you will (potentially) want to **update them** as the stream goes by.

To do so, **DON'T FETCH AGAIN**, it's not useful and not optimized. Instead, I created two nodes that will refresh the Data we got from twitch.

You can either chose the **“Refresh Stream Data”** to do a 3 in 1 fetch, covering “Stream / Channel / Follower”, or use the **“Refresh all Data”** to refresh everything twitch gave us.

→ Everytime you want the informations to refresh, call either one of them.

5. TwitchChat :



First of all, drop the *TwitchChat* component in your actor, I actually recommend putting all the twitch components in the same BP. I recommend setting the same parameters i did :

Connects to Twitch chat over WebSocket

Encrypts the connection

Used if WebSocket is OFF (Write : **irc.chat.twitch.tv**)

Used if WebSocket is OFF (use **6697**)

Used only when WebSocket is ON

(example: **wss://irc-ws.chat.twitch.tv:443/**)

DEBUGGING ONLY : Writes every raw IRC line to the log (very noisy → keep OFF in production)

What's those "**Max Queue Messages**" and "**Max Dispatch per Tick**" parameters ?

Chat can sometimes explode during raids. To protect your FPS, StreamSync queues raw IRC lines and releases a limited number each tick. You can control how big the queue is and how fast it's drained.

- Max Queue Messages :

It controls how many lines we can buffer. Bigger number means it will survive bigger spikes and use more memory. Start at 2000, raise if chat is huge.

- Max Dispatch per Tick :

Defines the maximum lines delivered to your Blueprint per frame. Bigger number means a lower latency (but more CPU per frame). Start at 200 and tune it based on your graph cost.

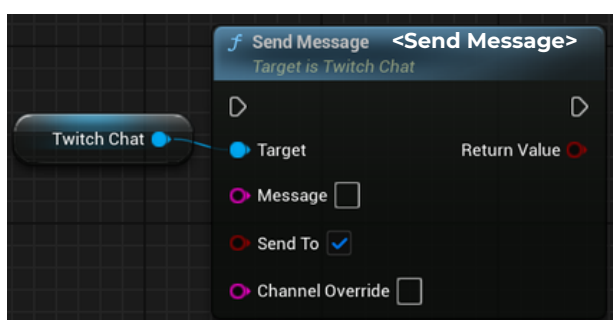
- Drop Oldest On Overflow :

It **defines what to throw away when the queue is Full** :

- **ON** : drop the oldest lines received (stay up to date → recommended for live)
- **OFF** : drop newest lines received (Keeps history, can make you lag behind)

Now that *Twitch Chat* is ready (**pls go see the Basic Setup page 4 if not done already**) :

Need to send a message in the chat ? :

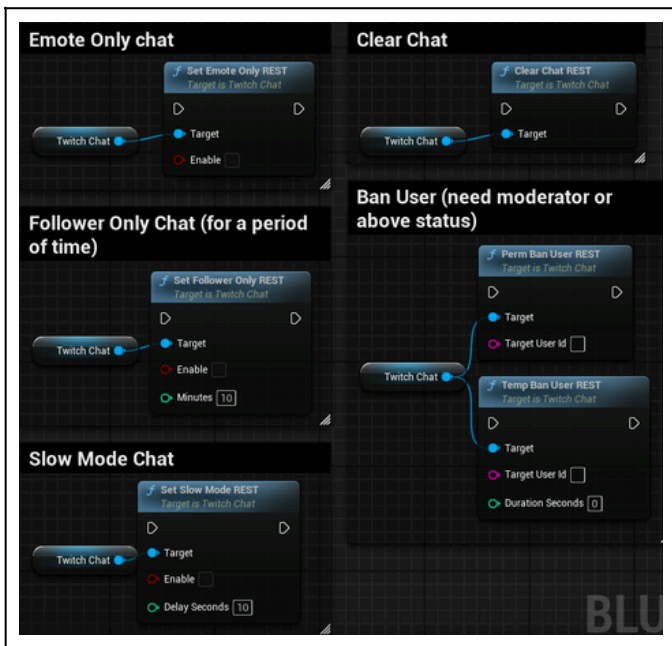


Just call the "**Send Message function**" (as simple as it is)

- If SendTo **True** : Sends a Casual Message
- If SendTo **False** : Sends a Command in chat

You can override the channel if needed, or let it empty to leave the default channel

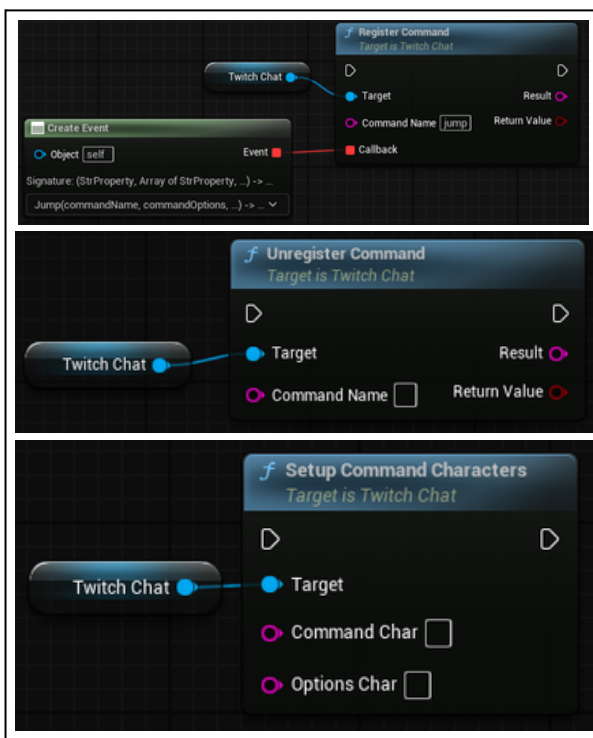
Twitch StreamSync



List of the functions you can call to interact with the chat in real-time :

- **Set Emote Only**
- **Clear Chat**
- **Set Follower Only**
- **Set Slow Mode**
- **Perm Ban User** (you'll need the user id)
- **Temp Ban User** (same here)

Need to make commands ? :



I'm sure you heard about commands in twitch chats, well you can now register your own commands.

Use the “**Register Command**” node with the command name you want. Don't add any prefix like “! / ? . ;” we'll deal with that just after.

You can then “**Create an Event**” or branch a “**Custom Event**” to the Callback, this will be the function that triggers when the command is written in the chat.

Use the “**Unregister Command**” to unregister any command you want.

Don't forget to “**Setup Command Characters**” : The goal here is to chose one prefix for our commands (example ! ? > ~ \$). By default set to “!”.

→ **AVOID : “/” and “.” since twitch treats those like client commands.**

“**Options Char**” are meant to split the arguments/options part of the command. By default it's just a “ ” (that's a space)

SPACE EXAMPLE : “!give item=sword qty=2”
COMMA EXAMPLE : “!give item=sword,qty=2”
PIPE EXAMPLE : “!give item=sword|qty=2”

Twitch StreamSync

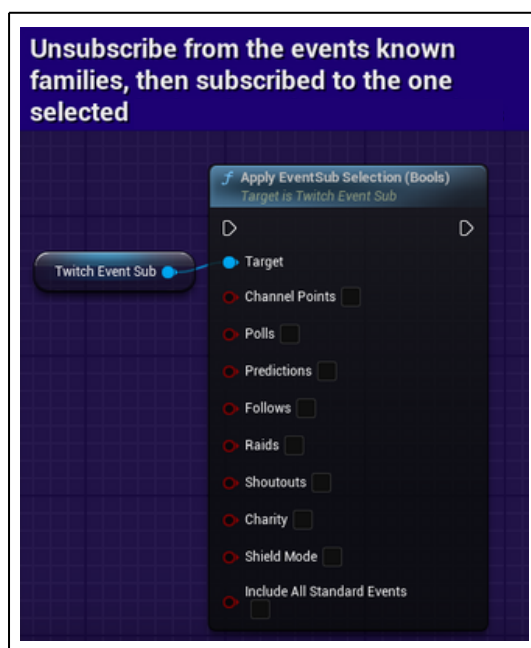
What about the events ? :

I wont go through all the events, they're pretty easy to understand and you can find them all either in the component, or in my demo blueprint <BP_StreamSyncDEMO> with a comment on each of them.

6. EventSub :

What is **EventSub** ?

→ It's a WebSocket or Webhook topic feed that allows us to react to events from twitch (redeems, bits, subs/gifts, raids, follows, polls, predictions, etc.) **WARNING : No subscription = no events !**



First of all, you will want to subscribe to the chosen events.

To do so, you must "**Apply EventSub Selection (Bools)**". You can now check all the topics you want to subscribe to. Note that "*Include All Standard Events*" means all the rest of the events from twitch (sub / cheer / ban / moderator etc.), it doesn't mean everything that is above.

Of course you can individually subscribe to any type by using the "**Subscribe <WhateverYouWant>**" nodes from event sub component, but I recommend using the easy and modular way explained above.

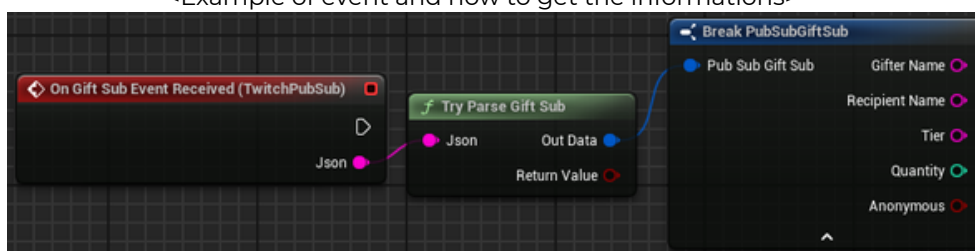
—————You're good to go—————

(To unsubscribe, just call "**Apply EventSub Selection Bools**" again and select only the ones you want to be subscribed to.

What about the **Events** ? :

Well, you can access them in the EventSub component or in my BP_SteamSyncDEMO :

<Example of event and how to get the informations>



As you can see, each event has his own "Parse", from which you can break the Data and get everything you need to know for the specific situation (user / Topic / view count etc.)

7. *Helix Update Stream :*

Requirements : The ClientID must match the token's app, and "broadcaster_id == token.user_id"
 → Do not worry, if you didnt do anything special (and if you followed the documentation) the requirements should be all met by themselves and it should be all good to go :)

The image displays several screenshots of the Twitch Helix API documentation for stream management functions:

- Set Stream Game:** Shows the 'Set Channel Game (Async)' function with parameters: Auth, Game Id, Game Name, On Success, and On Error.
- Set Stream Title:** Shows the 'Set Channel Title (Async)' function with parameters: Auth, New Title, On Success, and On Error.
- Create / Update Reward:** Shows two functions: 'Create Reward (Async)' and 'Update Reward (Async)'. 'Create Reward' includes parameters like Title, Cost, Prompt, Is Enabled, Is User Input Required, Is Global Cooldown Enabled, Global Cooldown Seconds, and Should Redemptions Skip Request Queue. 'Update Reward' includes parameters like Reward Id, New Title, New Cost, Set Cost, New Prompt, Set Prompt, New Enabled, Set Enabled, New User Input Required, and Set User Input Required.
- All the Polls functions:** Shows 'Create Poll (Async)' and 'End Poll (Async)'. 'Create Poll' includes parameters like Title, Choice Titles, Duration Seconds, Enable Channel Points Voting, and Channel Points Per Vote. 'End Poll' includes parameters like Poll Id and Choice Ids.
- All the predictions functions:** Shows four functions: 'Create Prediction (Async)', 'Lock Prediction (Async)', 'Resolve Prediction (Async)', and 'Cancel Prediction (Async)'. 'Create Prediction' includes parameters like Title, Outcome Titles, Prediction Window Seconds, and Outcome Ids. 'Lock Prediction' includes Prediction Id. 'Resolve Prediction' includes Prediction Id and Winning Outcome Id. 'Cancel Prediction' includes Prediction Id.

Here are the async functions you can call to interact with the stream :

- Set Channel Title (NEEDS SCOPE)
- Set Channel Game "channel:manage:broadcast"
- Create Reward (NEEDS SCOPE)
- Update Reward "channel:manage:redemptions"
- Create Poll (NEEDS SCOPE)
- End Poll "channel:manage:polls"
- Create Prediction (NEEDS SCOPE)
- Lock Prediction "channel:manage:predictions"
- Cancel Prediction
- Resolve Prediction

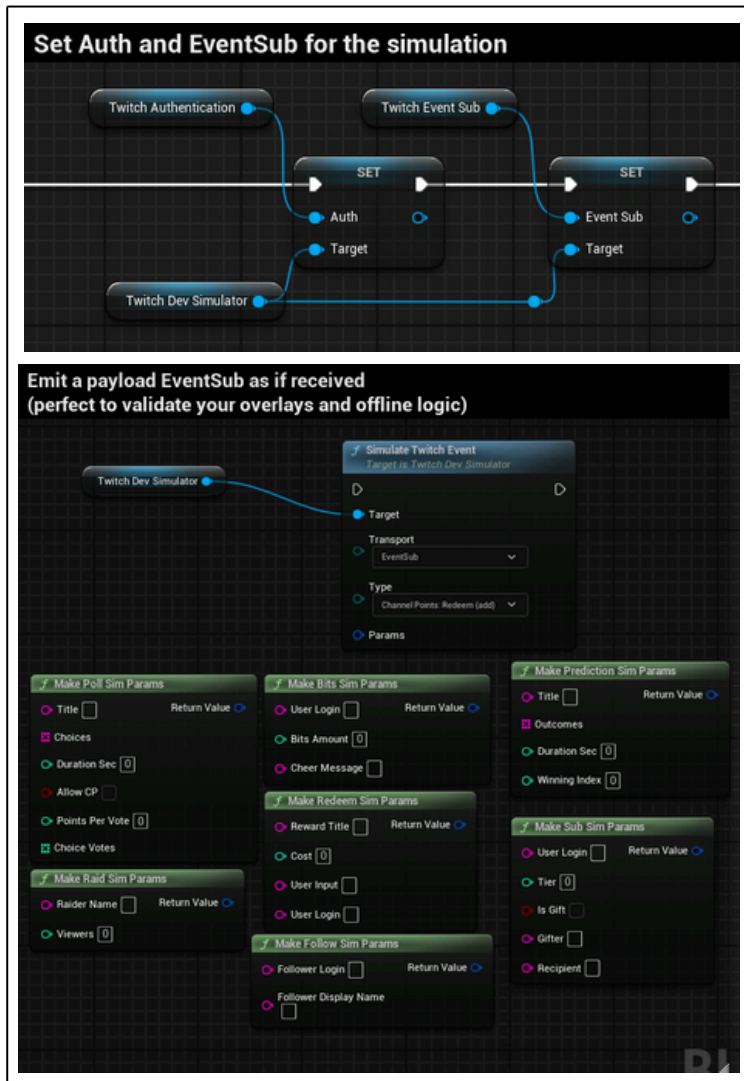
Common Pitfalls :

- Are you missing scopes ?
- Wrong ClientID
- Broadcaster Mismatch

8. *Dev Simulator :*

What's the purpose of *StreamSync's Dev Simulator*?

Well it's very simple, it allows you to **trigger Twitch-like events offline** to iterate UI/Gameplay without starting a stream.



FIRST OF ALL

→ Set the Auth and the EventSub in order for the Dev Sim to work properly. Without this it won't work at all

To simulate an Event, just call the “**Simulate Twitch Event**” from the *Dev Simulator* component. You then will have to chose some settings :

- **Type :**

This is the event you want to simulate. You can chose whatever you want.

- **Params :**

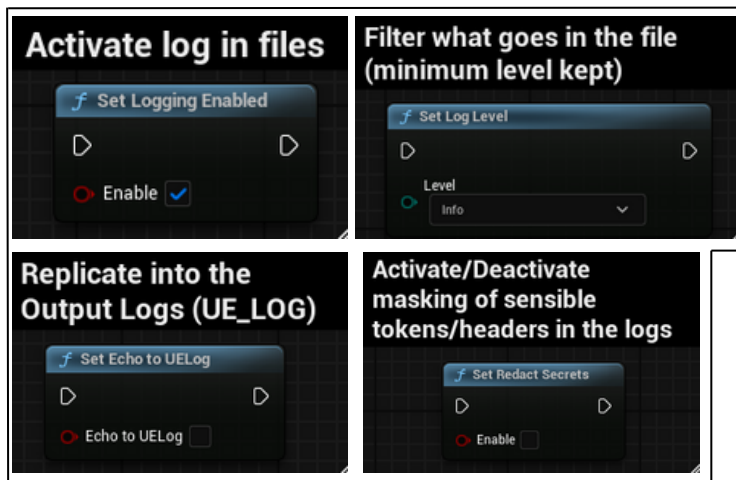
Chose the “Make <yourEvent> Sim Params> corresponding to the event you want to simulate. (There are plenty of them, chose wisely the one going with your event)

9. Twitch Log Exporter :

What's *StreamSync*'s **Twitch Log Exporter** ?

A lightweight logger you can turn on/off at runtime to capture human-readable lines, keyvalue metrics, and raw JSON from Chat/PubSub/EventSub/Helix in a text document. Secrets can be auto-redacted.

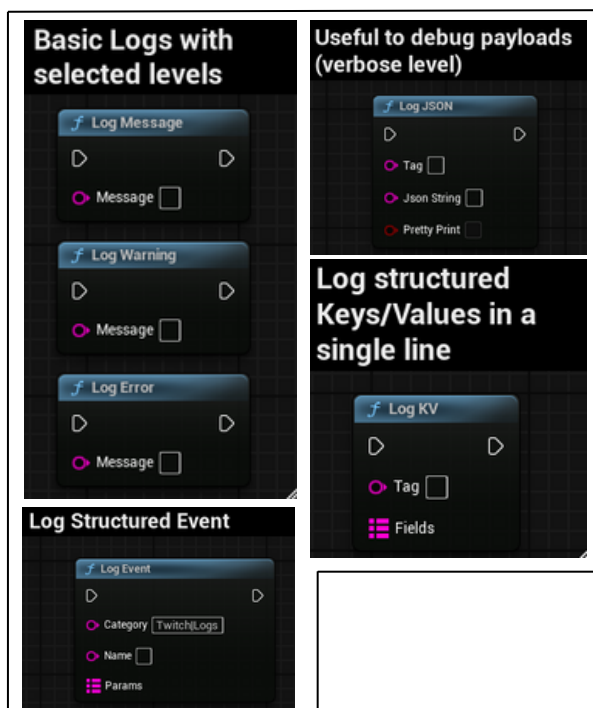
1. Setup (One-Time) :



Here are the global functions :

- **"SetLoggingEnabled"** toggles the exporter.
- **"SetLogLevel"** sets the minimum severity the logger will keep. Anything above is ignored (in the list).
- **"SetEchoToUE"** makes all the prints to duplicate in the *Unreal Output Log* (handy in PIE)
- **"SetRedactSecrets"** masks tokens, client secrets and sensitive IDs in all outputs. **KEEP THIS "ON" !**

2. Write Methods (Blueprint) :



Here are the logging functions :

- **"LogMessage"** logs a plain line for checkpoints.
- **"LogWarning"** is useful to log a notable issue that is not fatal (rate-limit nearing etc.)
- **"LogError"** is useful to log when something failed, paired with errors when available.
- **"LogEvent"** records a single named event with a set of properties (key values). Think of it as an analytics style row : one action, one payload, and level-tagged.
- **"LogJSON"** preserves the original payload (from PubSub/EventSub/Helix). Very useful for replay or debug.
- **"LogKV"** was made to log structured key-values. Great for Dashboards and diffing runs.

TIPS :)

PRODUCTION: SetLogLevel(**Info**), SetRedactSecrets(**true**), SetEchoToUELog(**false**)

Debug: SetLogLevel(**Verbose**), SetEchoToUELog(**true**) → for quick diagnosis.

10. *Webhook Transport (HTTPS callback) :*

TwitchStreamSync supports two EventSub transports: WebSocket and Webhook.

By default, the plugin automatically chooses WebSocket (the simplest mode), but you can also use Webhook when you need server-grade reliability and external callbacks.

1. Difference between WebSocket and Webhook :

WebSocket (Recommended for most users)

- The Unreal Engine client maintains an outgoing connection to Twitch.
- No public IP or server required : works perfectly inside the Editor or packaged builds.
- Ideal for games, plugins, and single-user tools.
- Simple, fast, and NAT-friendly.

Webhook (Advanced / Server-side)

- Twitch sends incoming HTTPS POST requests to a public callback URL that you host.
- Best suited for backend or dedicated server environments that are always online.
- Twitch automatically retries on failure and signs each payload using HMAC for integrity checks.
- Recommended if you manage multiple streamers, need centralized storage or analytics, or must receive events even when no client is running.

2. How Webhook works :

Instead of maintaining a live WebSocket connection to Twitch, **Twitch sends HTTPS POST requests to the plugin** whenever an event occurs (channel points redeemed, polls, predictions, raids, etc.).

Each POST includes a *JSON payload* and an *HMAC SHA-256 signature* (using your secret).

When the plugin receives a message, it automatically:

- **Replies to the initial challenge** request (the handshake Twitch uses to verify your callback).
- **Verifies the HMAC signature** with your WebhookSecret (so only Twitch can send data).
- **Parses and dispatches the EventSub event** to your Blueprints.

If verification fails, the plugin safely rejects the message.

If the local server fails to start or can't be reached, the plugin falls back to WebSocket automatically.

3. Enabling Webhook mode in TwitchStreamSync :

→ Open your **Project Settings** → **Twitch Sync Settings**

- Set **Use WebSocket** = **FALSE** (this enables the Webhook path).
- Set **Webhook Port** = *the local port to listen on (default: 8080)*.
- (Optional) Set Webhook Secret = any random string. Twitch uses this to sign each payload.
- (Optional) Customize the Local Webhook Path (default: /eventsub).

Twitch StreamSync

→ Provide a *Public Callback URL*. This is the public URL Twitch will call :

Exemple :

```
https://yourname.ngrok.io/eventsub
```

→ If you're testing locally, you can use a free tunnel tool such as Ngrok to expose your Unreal port:

```
ngrok http 8080
```

Copy the public HTTPS URL printed by Ngrok and paste it into PublicCallbackURL in the component or settings

→ Launch EventSub normally

- In your Blueprint, call **Initialize Twitch Integration** (with EventSub checked TRUE).
- The plugin detects **bUseWebhooks == true**, verifies that your engine supports the HTTP server, and starts the local listener with *StartLocalWebhookServer()* (c++)
- It will then automatically register your Webhook subscription to Twitch using your PublicCallbackURL and WebhookSecret.

If you don't need a backend or public server, keep WebSocket, it's simpler doesn't need anything else than what the plugin provides.

Use Webhook when you want server-grade reliability or central event handling.

11. Refresh Token :

What is an OAuth Token? ?

When you connect to Twitch, the plugin uses **OAuth 2.0** to obtain an access token.

This token represents the **user's authorization** and is used on every request to Twitch APIs (Helix, EventSub, etc.).

→ It's like a **digital key**: it proves that the player allowed your game to act on their behalf (read chat, update title, create rewards...).

Each token has:

- **Access Token** → short-lived (usually valid for a few hours).
- **Refresh Token** → long-lived, used to silently renew the access token when it expires.

Without the refresh mechanism, users would need to manually log in every few hours.

What about the way we handle the Token refreshing in StreamSync ?

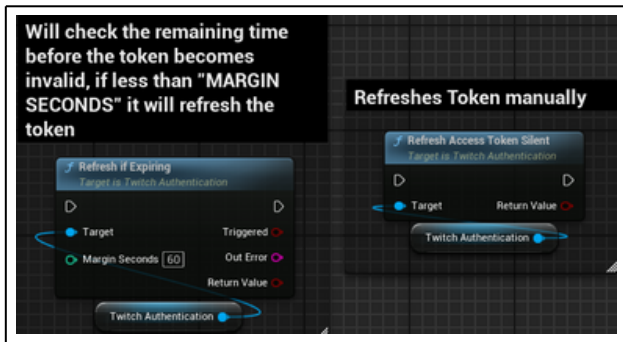
StreamSync constantly checks the expiration time stored inside the Authentication component.

- Each token comes with an *expires_in* value.
- This is converted into *AccessTokenExpiryUtc*.
- When it's about to expire, the plugin calls **RefreshAccessTokenSilent()** automatically.

→ **No browser opens, no pop-up, no interruption.**

Everything happens silently in the background through the built-in HTTP request to Twitch.

Twitch StreamSync



If you ever want to refresh the token yourself (for example, after a long pause or when resuming a session, you can call "**Refresh Access Token Silent**".

If you want, you can create a function that calls "**Refresh if Expiring**" every 30 seconds. This function will refresh the token if its margin seconds before expiring is less than <Margin Seconds> (just in case)

12. Error Handling :

Step 1 - Identify the channel of failure :

- Helix (HTTP)

You see an HTTP status code (e.g., 200/204/400/401/403/429/5xx), optional response JSON, and a route you called (/channels, /predictions, etc.). The error surfaces on the node's `OnError` (async callback) or your HTTP wrapper.

Tip: Helix fails even if sockets are fine.

- Socket (Chat / PubSub / EventSub)

There is no HTTP status code. You see connect/disconnect, PING/PONG or keepalive messages, LISTEN ACK (PubSub), or welcome (EventSub). Errors surface on each component's `OnError` or as server notices.

Step 2 - Quick checks per channel :

- **Helix** (writes like Title/Category/Rewards/Polls/Predictions)

1. Run `ValidateOAuthToken` → confirm `user_id` and required scopes.
2. Ensure token is a user token for the broadcaster (`broadcaster_id == token.user_id`).
3. Ensure Client-Id matches the app that minted the token.
4. 429 → backoff; log rate-limit headers (limit, remaining, reset).
5. For game/category set, prefer `GamedId` over name lookups.

- **Chat** (IRC WS/TCP)

1. Connect after auth; wait for `OnChatConnected`.
2. Can't send? Add `chat:edit`; ensure you joined the channel; check server NOTICE (slow/emote/followers-only).
3. Frequent drops → enable TLS/WS; verify no flood (pace messages).

- **EventSub** (WS/Webhook)

1. WS: expect welcome then periodic keepalives. If quiet, you likely didn't create subscriptions → call `ApplyEventSubSelection` and `ListSubscriptions`.
2. Webhook: must have a public HTTPS URL and correct secret; signature/HMAC failures appear as verify errors.
3. Duplicate events? You might be consuming the same family via PubSub and EventSub—pick one.

Twitch StreamSync

Step 3 - Common symptoms → quick fixes :

- Title/Category returns 401/403 → add channel:manage:broadcast; use the broadcaster's user token; fix Client-Id mismatch.
 - Polls/Predictions fail → add channel:manage:polls or channel:manage:predictions.
 - Reward create/update fails → add channel:manage:redemptions.
 - PubSub silent → no LISTEN ACK (you didn't subscribe) or PING/PONG stalled.
 - EventSub silent → no subscriptions created, or Webhook URL/secret invalid.
 - Double triggers → enforce one transport per family (either EventSub or PubSub).
-

What to log so support is easy :

- Helix: status, route, rate-limit headers, response body.
- Sockets: last keepalive / PING time, reconnect attempts, active topics/subscriptions.
- Permissions: expected scopes vs actual token scopes (use your scope helpers).

Safe defaults :

- Production: LogLevel=Info, RedactSecrets=ON, minimal echo to UE log.
- Debug: LogLevel=Verbose, enable LogJSON briefly, then revert.

13. Annexes :

Difference between WebSocket and Webhook :

WebSocket (Recommended for most users)

- The Unreal Engine client maintains an outgoing connection to Twitch.
- No public IP or server required — works perfectly inside the Editor or packaged builds.
- Ideal for games, plugins, and single-user tools.
- Simple, fast, and NAT-friendly.

Webhook (Advanced / Server-side)

- Twitch sends incoming HTTPS POST requests to a public callback URL that you host.
- Best suited for backend or dedicated server environments that are always online.
- Twitch automatically retries on failure and signs each payload using HMAC for integrity checks.
- Recommended if you manage multiple streamers, need centralized storage or analytics, or must receive events even when no client is running.

Need Informations on the scopes ?

<https://dev.twitch.tv/docs/eventsub/eventsub-subscription-types>